# Coalesced memory access and device (GPU) memory types
# Electronics-ICT

(B-KUL-JLI1QS)

Arne Vanmarcke, Thibault Dekock

March 9, 2025




Instructor: Thomas Feys

# 1 Introduction

In this lab, we analyze the impact of memory access patterns on GPU performance. Part 1 measures execution times for coalesced and uncoalesced memory access, demonstrating how efficient memory access can significantly improve performance. In part 2, we implement matrix multiplication using different memory types: global, shared, and constant memory. We compare their execution times and evaluate the impact of memory migration between the CPU and GPU. Through these experiments, we gain a deeper understanding of memory optimization techniques and their practical implications in high-performance computing.

# 2 Solution

## 2.1 Coalesced memory access

The performance difference between coalesced and uncoalesced memory access can be seen on figure 1. We see that the coalesced memory access is faster for smaller images but when the image size drastically increases, uncoalesced memory access yields better results.

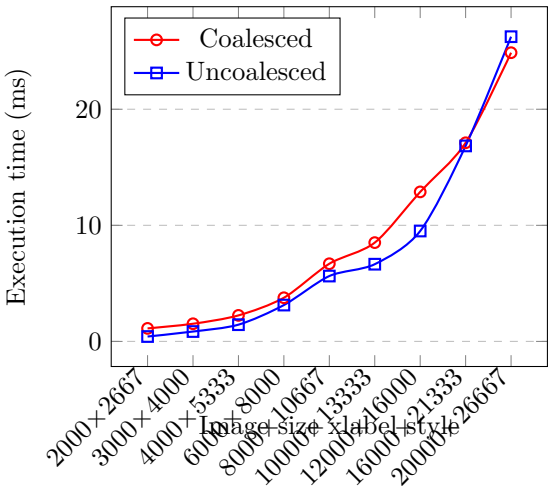



Figure 1: Execution time coalesced vs uncoalesced memory access

### 2.1.1 Resulting images

As required, the red color component of the RGB value in the image is inverted.



(a) unprocessed image



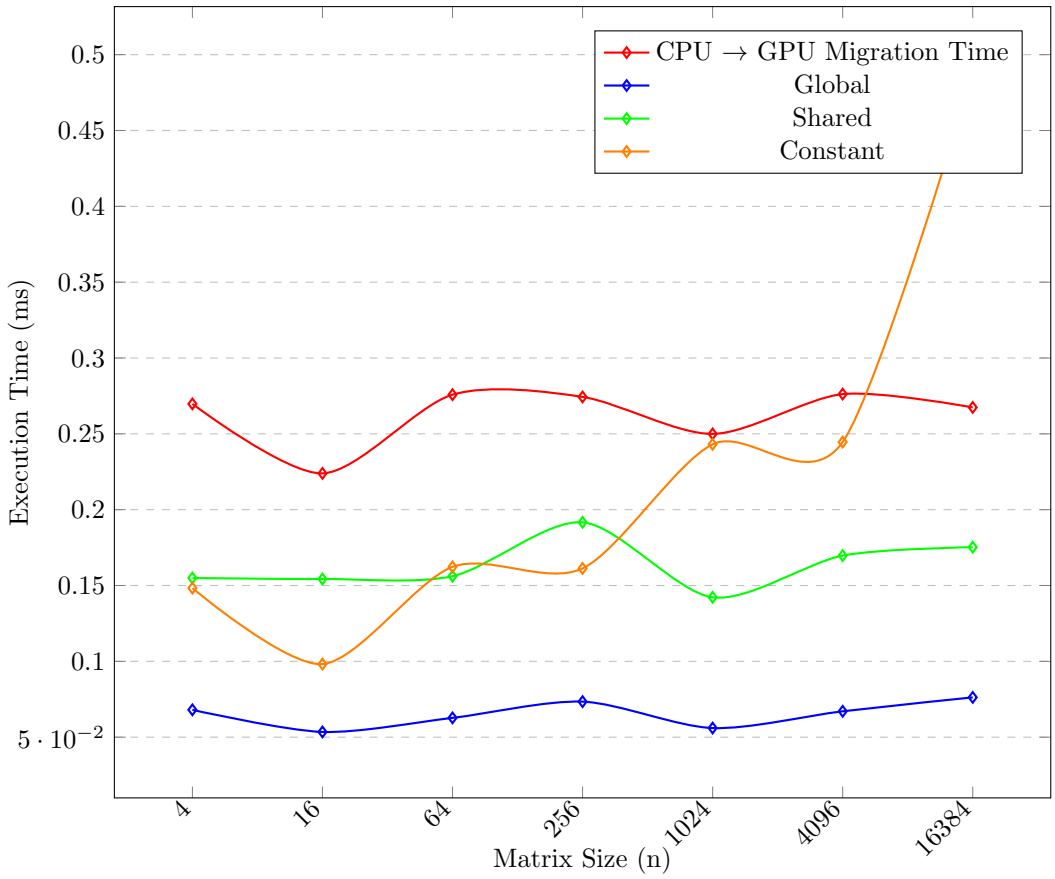
(b) processed image

## 2.2 Device (GPU) memory types

Based on the average execution times over 100 runs shown in the table below, we observe that most execution times remain relatively consistent. The execution times using constant memory shows more or less increase as the matrix size grows. Additionally, the results indicate that utilizing shared memory is consistently slower than using global memory. The execution times for constant memory fall between these two: it outperforms shared memory for smaller matrix sizes but becomes slower than both shared and global memory as the matrix size increases.

| Matrix Size (n) | CPU → GPU Time (ms) | Global (ms) | Shared (ms) | Constant (ms) |
|---|---|---|---|---|
| 4 | 0.269753 | 0.0680339 | 0.154961 | 0.148292 |
| 16 | 0.223914 | 0.053448 | 0.154304 | 0.0981853 |
| 64 | 0.275834 | 0.0627213 | 0.156132 | 0.1623 |
| 256 | 0.274347 | 0.0734246 | 0.191686 | 0.16127 |
| 1024 | 0.249968 | 0.0560218 | 0.142221 | 0.243102 |
| 4096 | 0.27621 | 0.0670544 | 0.169726 | 0.244614 |
| 16384 | 0.267432 | 0.0762407 | 0.175248 | 0.488211 |

Table 1: Execution Times for Different Matrix Sizes over 100 Runs



## 2.3 Code

### 2.3.1 Coalesced memory access

```
#include <cuda_runtime.h>
#include <iostream>
#include <cstdint>
#include <cstring>

#define M 960                       // Image width
#define N 1280                     // Image height
#define C 3          // Colors
#define BLOCK_SIZE 32

__global__ void invertRedCoalesced(unsigned char *img, int width, int height) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height;

    for (int i = idx; i < totalPixels; i += stride) {
        img[i] = 255 - img[i];
    }
}

__global__ void invertRedUncoalesced(unsigned char *img, int width, int height) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height;

    for (int i = idx; i < totalPixels; i += stride) {
        img[i*C] = 255 - img[i*C];
    }
}

void convertToPlanar(unsigned char *input, unsigned char *output, int width, int
    height) {
    int totalPixels = width * height;
    int redOffset = 0;
    int greenOffset = totalPixels;
    int blueOffset = 2 * totalPixels;

    for (int i = 0; i < totalPixels; i++) {
        output[redOffset + i] = input[i * C];
        output[greenOffset + i] = input[i * C + 1];
        output[blueOffset + i] = input[i * C + 2];
    }
}
void convertToInterleaved(unsigned char *input, unsigned char *output, int width,
    int height) {
    int totalPixels = width * height;
    int redOffset = 0;
    int greenOffset = totalPixels;
    int blueOffset = 2 * totalPixels;

    for (int i = 0; i < totalPixels; i++) {
        output[i * C] = input[redOffset + i];
        output[i * C + 1] = input[greenOffset + i];
        output[i * C + 2] = input[blueOffset + i];
    }
}

void checkCudaError(cudaError_t err, const char *msg) {
```

```
    if (err != cudaSuccess) {
        std::cerr << msg << " (" << cudaGetErrorString(err) << ")\n";
        exit(EXIT_FAILURE);
    }
}

void processImageCUDA(unsigned char *h_img, int width, int height, int numThreads,
     bool coalesced) {
    int totalPixels = width * height * C;
    int imgSize = totalPixels * sizeof(unsigned char);
    unsigned char *d_img;

    checkCudaError(cudaMalloc((void **)&d_img, imgSize), "Failed to allocate
    device memory");
    checkCudaError(cudaMemcpy(d_img, h_img, imgSize, cudaMemcpyHostToDevice), "
    Failed to copy image to device");

    int numBlocks = (totalPixels / C + numThreads - 1) / numThreads;

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    if (coalesced) {
        invertRedCoalesced<<<numBlocks, numThreads>>>(d_img, width, height);
    } else {
        invertRedUncoalesced<<<numBlocks, numThreads>>>(d_img, width, height);
    }
    cudaEventRecord(stop);

    checkCudaError(cudaGetLastError(), "Kernel execution failed");
    checkCudaError(cudaMemcpy(h_img, d_img, imgSize, cudaMemcpyDeviceToHost), "
    Failed to copy result to host");

    cudaEventSynchronize(stop);
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    std::cout << "Execution time: " << milliseconds << " ms\n";

    cudaFree(d_img);
}

int main() {
    uint8_t* image = (uint8_t*)malloc(M * N * C * sizeof(uint8_t));
    uint8_t* planar_image = (uint8_t*)malloc(M * N * C * sizeof(uint8_t));

    // init array
    for (int i = 0; i < M * N * C; i++) {
        image[i] = 150;
    }

    convertToPlanar(image, planar_image, M, N);
    std::cout << "Image size: " << M << "x" << N << "x" << C << "\n";
    for (int numThreads = 1024; numThreads <= 1024; numThreads *= 2) {
        std::cout << "Running coalesced with " << numThreads << " threads...\n";
        processImageCUDA(planar_image, M, N, numThreads, true);

        std::cout << "Running uncoalesced with " << numThreads << " threads...\n";
        processImageCUDA(image, M, N, numThreads, false);
    }

```

```
115     FILE *outputFile = fopen("./output_image.ppm", "wb");
116     fprintf(outputFile, "P6\n%d %d\n255\n", M, N);
117     fwrite(image, 1, M * N * C, outputFile);
118     fclose(outputFile);
119 
120     convertToInterleaved(planar_image, image, M, N);
121 
122     FILE *outputFilePlanar = fopen("./output_image_planar.ppm", "wb");
123     fprintf(outputFilePlanar, "P6\n%d %d\n255\n", M, N);
124     fwrite(image, 1, M * N * C, outputFilePlanar);
125     fclose(outputFilePlanar);
126 
127     free(planar_image);
128     free(image);
129     return 0;
130 }
```

### 2.3.2 Device memory types

```cpp
#include <cuda_runtime.h>
#include <iostream>
#include <iomanip>

#define BLOCK_SIZE 16
#define MATRIX_SIZE 4

using namespace std;

// Global memory
__global__ void matMulGlobal(float *A, float *B, float *C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

// Shared memory
__global__ void matMulShared(float *A, float *B, float *C, int N) {
    __shared__ float Asub[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ float Bsub[BLOCK_SIZE][BLOCK_SIZE];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0;

    for (int k = 0; k < (N + BLOCK_SIZE - 1) / BLOCK_SIZE; k++) {
        Asub[threadIdx.y][threadIdx.x] = A[row * N + (k * BLOCK_SIZE + threadIdx.x
    )];
        Bsub[threadIdx.y][threadIdx.x] = B[(k * BLOCK_SIZE + threadIdx.y) * N +
    col];
        __syncthreads();
        for (int i = 0; i < BLOCK_SIZE; i++) {
            sum += Asub[threadIdx.y][i] * Bsub[i][threadIdx.x];
        }
        __syncthreads();
    }
    C[row * N + col] = sum;
}

// Constant memory
__constant__ float B_const[MATRIX_SIZE * MATRIX_SIZE];
__global__ void matMulConstant(float *A, float *C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B_const[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

int main() {
```

```
59      cout << "Matrix size " << MATRIX_SIZE * MATRIX_SIZE << endl;
60
61      float A[MATRIX_SIZE * MATRIX_SIZE];
62      float B[MATRIX_SIZE * MATRIX_SIZE];
63      float C[MATRIX_SIZE * MATRIX_SIZE] = {0};
64
65      for(int i = 0; i<MATRIX_SIZE * MATRIX_SIZE; i++){
66          A[i] = 2;
67          B[i] = 3;
68      }
69
70      float *d_A, *d_B, *d_C;
71      cudaMalloc(&d_A, MATRIX_SIZE * MATRIX_SIZE * sizeof(float));
72      cudaMalloc(&d_B, MATRIX_SIZE * MATRIX_SIZE * sizeof(float));
73      cudaMalloc(&d_C, MATRIX_SIZE * MATRIX_SIZE * sizeof(float));
74
75      float milliseconds = 0;
76      cudaEvent_t startEvent, stopEvent;
77      cudaEventCreate(&startEvent);
78      cudaEventCreate(&stopEvent);
79
80      cudaEventRecord(startEvent);
81      cudaMemcpy(d_A, A, MATRIX_SIZE * MATRIX_SIZE * sizeof(float),
        cudaMemcpyHostToDevice);
82      cudaMemcpy(d_B, B, MATRIX_SIZE * MATRIX_SIZE * sizeof(float),
        cudaMemcpyHostToDevice);
83      cudaEventRecord(stopEvent);
84      cudaEventSynchronize(stopEvent);
85      cudaEventElapsedTime(&milliseconds, startEvent, stopEvent);
86      cout << "Time for CPU to GPU global memory migration: " << milliseconds << "
        ms" << endl;
87
88      cudaMemcpyToSymbol(B_const, B, MATRIX_SIZE * MATRIX_SIZE * sizeof(float));
89
90      dim3 threadsPerBlock(BLOCK_SIZE, BLOCK_SIZE);
91      dim3 blocksPerGrid((MATRIX_SIZE + BLOCK_SIZE - 1) / BLOCK_SIZE, (MATRIX_SIZE +
         BLOCK_SIZE - 1) / BLOCK_SIZE);
92
93
94
95      // Global Memory
96      cudaEventRecord(startEvent);
97      matMulGlobal<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, MATRIX_SIZE);
98      cudaDeviceSynchronize();
99      cudaEventRecord(stopEvent);
100     cudaEventSynchronize(stopEvent);
101     cudaEventElapsedTime(&milliseconds, startEvent, stopEvent);
102     cout << "Execution time (Global): " << milliseconds << " ms" << endl;
103
104     cudaMemcpy(C, d_C, MATRIX_SIZE * MATRIX_SIZE * sizeof(float),
        cudaMemcpyDeviceToHost);
105
106
107     // Shared Memory
108     cudaEventRecord(startEvent);
109     matMulShared<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, MATRIX_SIZE);
110     cudaDeviceSynchronize();
111     cudaEventRecord(stopEvent);
112     cudaEventSynchronize(stopEvent);
113     cudaEventElapsedTime(&milliseconds, startEvent, stopEvent);
114     cout << "Execution time (Shared): " << milliseconds << " ms" << endl;
115
```

```
116     cudaMemcpy(C, d_C, MATRIX_SIZE * MATRIX_SIZE * sizeof(float),
        cudaMemcpyDeviceToHost);
117
118     // Constant Memory
119     cudaEventRecord(startEvent);
120     matMulConstant<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_C, MATRIX_SIZE);
121     cudaDeviceSynchronize();
122     cudaEventRecord(stopEvent);
123     cudaEventSynchronize(stopEvent);
124     cudaEventElapsedTime(&milliseconds, startEvent, stopEvent);
125     cout << "Execution time (Constant): " << milliseconds << " ms" << endl;
126
127     cudaMemcpy(C, d_C, MATRIX_SIZE * MATRIX_SIZE * sizeof(float),
        cudaMemcpyDeviceToHost);
128
129     // cout << "Result Matrix:\n";
130     // for (int i = 0; i < MATRIX_SIZE; i++) {
131     //     for (int j = 0; j < MATRIX_SIZE; j++) {
132     //         cout << setw(2) << C[i * MATRIX_SIZE + j] << " ";
133     //     }
134     //     cout << endl;
135     // }
136
137     cudaFree(d_A);
138     cudaFree(d_B);
139     cudaFree(d_C);
140
141     return 0;
142 }
```